

What to automate: ParaBank application

1. What to automate - prioritization

For a new project (ParaBank in this case), I would start first with happy paths and critical flows, followed by a regression suite. Lets detail this a little bit:

Smoke (happy paths)

Critical flows that are vital for our app, and starting with the most common user cases, to move in the future to negative and edge cases.

- Registration
- Authentication
- Core transactions (Fund transfer and Bill Pay happy paths)

Regression

Any repetitive tests valuable for our application, or areas bug prone that were already problematic in the past and are always checked in sanity checks.

2. What to NOT automate

Even if we can automate and validate our application visuals, exact CSS pixel checks are usually fragile. We can do this for static pages and even use screenshot comparison, but I would leave this for very specific areas.

Also we need to keep in mind that a big part of the value in automating tests is their reusability, so we should avoid one time tests for really specific features that hardly will change, and are not vital for our system (I.e.: some temporary feature or visual change, or some very specific feature used by a low amount of clients. We can check this with the Product team to establish priorities).

3. Automation good practices

Flakiness

As a rule of thumb, we should always aim for an idempotent test that can run on its own. For this we would always try to have independent test data, or setting appropriate before & after hooks to ensure the test is always run in the same conditions.

Also using modern frameworks like Playwright allows us to be smart with auto-waitings (instead of hard waits), based on UI components or backend api calls. We should combine this with smart and realistic timeouts per action and retries.

Reliable and Scalable

For reliability, besides the control on the data used during the test execution, we should also aim for a reliable and stable environment.

Observability of test results is also very important, to quickly identify why a pipeline is “red”, or even to showcase what is being tested and document the flows to be understood by anyone (gherkin, screenshots, Playwright trace, pipeline reports).

Execution time is also important: a really slow test or pipeline is usually more error prone and we lose the advantage of a quick check VS manual testing. We should always aim to check what really matters, and to make our code efficient and fast.

For further details on the project architecture, please check Part 3 document.